

CHP#15: OPERATORS & EXPRESSIONS

- **Topics Which We Will Cover in this Chapter:**

- Operators and its Classification
- Conversion of Data type & type casting
- Some common Library Functions

Topic#01:

Operators and its Classification

CHP#15: OPERATORS & EXPRESSIONS

- **Operator:**

Operators are symbols that perform operations on variables and values.

Example: +, -, * etc

CHP#15: OPERATORS & EXPRESSIONS

- **Operator:**

Operators are symbols that perform operations on variables and values.

Example: +, -, * etc

- **Operands:**

Operands are the values or variables that operators act upon.

For example, in the expression $5 + 3$:

CHP#15: OPERATORS & EXPRESSIONS

- **Operator:**

Operators are symbols that perform operations on variables and values.

Example: +, -, * etc

- **Operands:**

Operands are the values or variables that operators act upon.

For example, in the expression $5 + 3$:

- **Expression:**

Expressions are combinations of operators and operands that are evaluated to produce a new value. An expression can consist of a single value or variable, or it can be a more complex combination of multiple values, variables, and operators.

CHP#15: OPERATORS & EXPRESSIONS

- **Operator:**

Operators are symbols that perform operations on variables and values.

Example: +, -, * etc

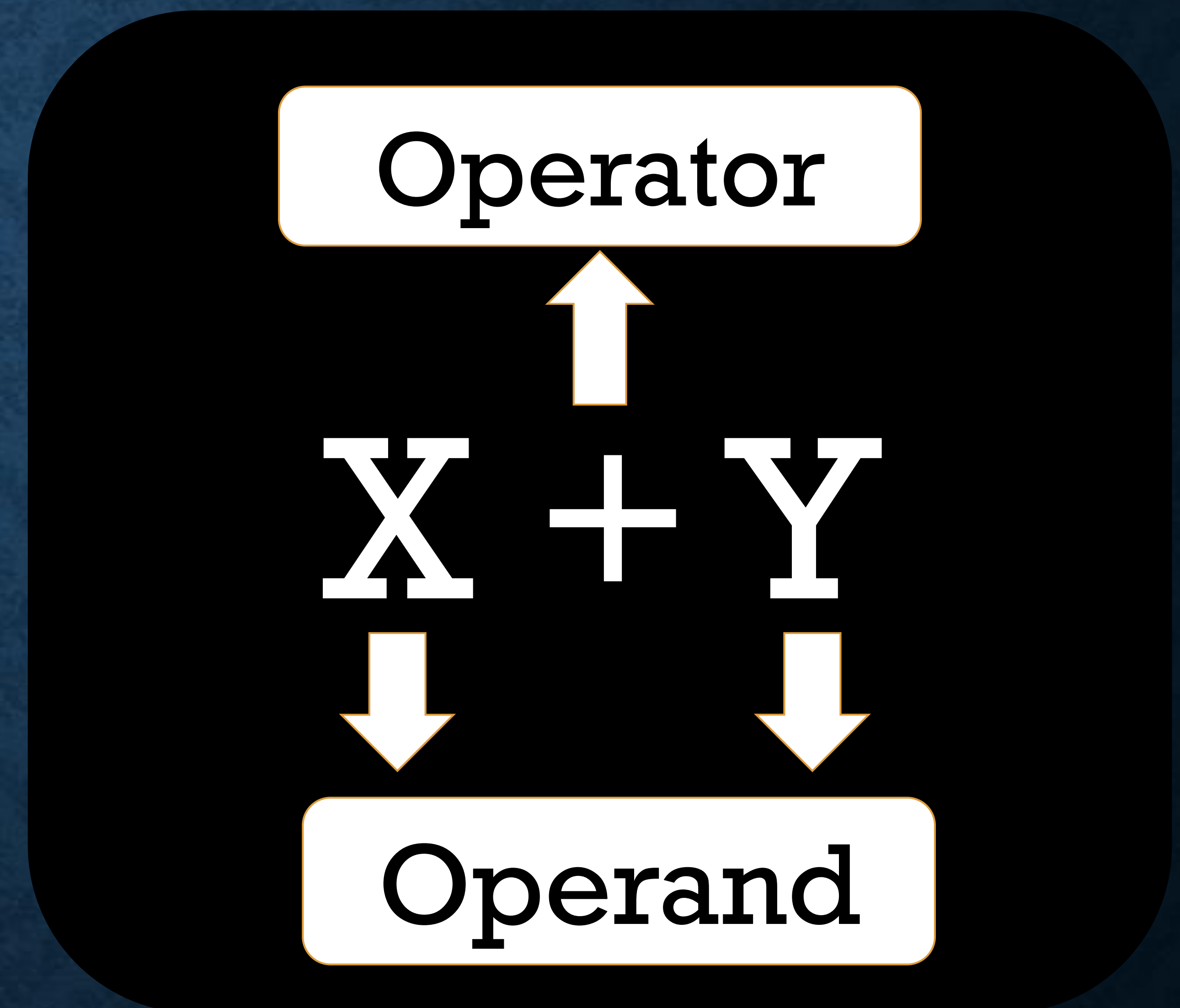
- **Operands:**

Operands are the values or variables that operators act upon.

For example, in the expression $5 + 3$:

- **Expression:**

Expressions are combinations of operators and operands that are evaluated to produce a new value. An expression can consist of a single value or variable, or it can be a more complex combination of multiple values, variables, and operators.



CHP#15: OPERATORS & EXPRESSIONS

- **Classification of Operators:**



CHP#15: OPERATORS & EXPRESSIONS

- **Commonly Used Operators are:**

1. Arithmetic Operators
2. Assignment Operators
3. Relational Operators
4. Logical/Boolean Operators
5. Increment & Decrement Operators

CHP#13: ALGORITHM & FLOWCHARTS

Any Doubt?

CHP#15: OPERATORS & EXPRESSIONS

Arithmetic Operators		Let us assume X and Y are two variables
Operator	Expression	Description
+	$X + Y$	To perform addition
-	$X - Y$	To perform subtraction
*	$X * Y$	To perform multiplication
/	X / Y	To perform division
%	$X \% Y$	To perform modulus

CHP#15: OPERATORS & EXPRESSIONS

Assignment Operator

Assignment operators are used to assigning value to a variable.

x	=	y
---	---	---

x is assign with value of y

x	+=	y
---	----	---

Equivalent to, $x = x + y$

x	-=	y
---	----	---

Equivalent to, $x = x - y$

x	*=	y
---	----	---

Equivalent to, $x = x * y$

x	/=	y
---	----	---

Equivalent to, $x = x / y$

CHP#15: OPERATORS & EXPRESSIONS

Relational Operators		Suppose X and Y are two variables
Operator	Expression	Description
<	$X < Y$	X is less than Y
<=	$X \leq Y$	X is less than or equal to Y
>	$X > Y$	X is greater than Y
>=	$X \geq Y$	X is greater than or equal to Y
==	$X == Y$	X is equal to Y
!=	$X != Y$	X is not equal to Y

CHP#15: OPERATORS & EXPRESSIONS

Logical Operators		
Operator	Description	Example
&&	AND	x=6 y=3 x<10 && y>1 Return True
 	OR	x=6 y=3 x==5 y==5 Return False
!	NOT	x=6 y=3 !(x==y) Return True

CHP#15: OPERATORS & EXPRESSIONS

Increment/Decrement Operators		Let us assume X is a variable
Operator	Expression	Description
++	++X	Pre-increment
	X++	Post-increment
--	--X	Pre-decrement
	X--	Post-decrement

CHP#15: OPERATORS & EXPRESSIONS

- **Difference between == and = Operator:**
- **Assignment operator (=)** → used to store a value into a variable.
 - Assigns the **right-hand side value** to the **left-hand side variable**.

```
int a;  
a = 10; // Assigns 10 to variable a
```

- **Equality Operator (==)** → used to **compare two values**.
 - Compares **two values or expressions** and checks if they are **equal**.

```
if (a == 10)  
    printf("Equal");
```

CHP#13: ALGORITHM & FLOWCHARTS

Any Doubt?

CHP#15: OPERATORS & EXPRESSIONS

Operators	Type of Operator	Associativity
++ -- ! Cast	Unary	Right to Left
* / %	Multiplicative	Left to Right
+ -	Additive	Left to Right
< > <= >=	Relational	Left to Right
== !=	Equality	Left to Right
&&	Logical AND	Left to Right
 	Logical OR	Left to Right
= *= %= += -=	Assignment	Left to Right

CHP#15: OPERATORS & EXPRESSIONS

- **Practice Questions (P.Q):**

1. **PQ#01:** `int result = 10 + 3 * 2 - 4 / 2;`

2. **PQ#02:** `int result = 3 + 4 * 2 / (1 - 5) % 2;`

CHP#13: ALGORITHM & FLOWCHARTS

Any Doubt?

CHP#15: OPERATORS & EXPRESSIONS

- **Lecture Task (LT):**

1. `int result = 10 * (6 / 2 + 5 % 3) - 4 * 2;`
2. `int result = (12 + 3 * 2) / (2 + 1) - (5 % 2 + 1);`

Do it now

CHP#15: OPERATORS & EXPRESSIONS

- **Lecture Task (LT):**

1. `int result = 10 * (6 / 2 + 5 % 3) - 4 * 2;`
2. `int result = (12 + 3 * 2) / (2 + 1) - (5 % 2 + 1);`

1.result = 42

2. result = 4

Topic#02:

Conversion of Data type & type casting

CHP#15: OPERATORS & EXPRESSIONS

- **Conversions of Data Type:**

Supposing, we have to solve this expressions (I + j).

First Operand	Second Operand	Result
Int	Int	Int
Int	Float	Float
Int	Char	Int
Int	Long int	Long int
Int	Double	double

Double	8 Bytes
Int	2 or 4 Bytes
Float	4 Bytes
Long int	4 Bytes
Char	1 Bytes

CHP#15: OPERATORS & EXPRESSIONS

- **Conversions of Data Type:**

Supposing, we have to solve this expressions (I + j).

First Operand	Second Operand	Result
Char	Int	Int
Char	Float	Float
Char	Char	Char
Char	Long int	Long int
Char	Double	double

Double	8 Bytes
Int	2 or 4 Bytes
Float	4 Bytes
Long int	4 Bytes
Char	1 Bytes

CHP#15: OPERATORS & EXPRESSIONS

- **Conversions of Data Type:**

Supposing, we have to solve this expressions (I + j).

First Operand	Second Operand	Result
Float	Int	Float
Float	Float	Float
Float	Char	Int
Float	Long int	Float
Float	Double	double

Double	8 Bytes
Int	2 or 4 Bytes
Float	4 Bytes
Long int	4 Bytes
Char	1 Bytes

CHP#15: OPERATORS & EXPRESSIONS

- **Conversions of Data Type:**

Supposing, we have to solve this expressions (I + j).

First Operand	Second Operand	Result
Long int	Int	Long int
Long int	Float	Float
Long int	Char	Long int
Long int	Long int	Long int
Long int	Double	Double

Double	8 Bytes
Int	2 or 4 Bytes
Float	4 Bytes
Long int	4 Bytes
Char	1 Bytes

CHP#15: OPERATORS & EXPRESSIONS

- **Conversions of Data Type:**

Supposing, we have to solve this expressions (I + j).

First Operand	Second Operand	Result
double	Int	double
double	Float	double
double	Char	double
double	Long int	double
double	Double	double

Double	8 Bytes
Int	2 or 4 Bytes
Float	4 Bytes
Long int	4 Bytes
Char	1 Bytes

CHP#13: ALGORITHM & FLOWCHARTS

Any Doubt?

CHP#15: OPERATORS & EXPRESSIONS

- **Conversions of Data Type:**

Supposing, we have to solve this expressions (I + j).

Do it now

Operation	Result	Operation	Result
5/2		2 / 5	
5.0 / 2		2.0 / 5.0	
5 / 2.0		2 / 5.0	
5.0 / 2.0		2.0 / 5.0	

Double	8 Bytes
Int	2 or 4 Bytes
Float	4 Bytes
Long int	4 Bytes
Char	1 Bytes

CHP#15: OPERATORS & EXPRESSIONS

- **Conversions of Data Type:**

Supposing, we have to solve this expressions (I + j).

Operation	Result	Operation	Result
5/2	2	2 / 5	0
5.0 / 2	2.5	2.0 / 5.0	0.4
5 / 2.0	2.5	2 / 5.0	0.4
5.0 / 2.0	2.5	2.0 / 5.0	0.4

Double	8 Bytes
Int	2 or 4 Bytes
Float	4 Bytes
Long int	4 Bytes
Char	1 Bytes

CHP#15: OPERATORS & EXPRESSIONS

- **Type Casting:**

Typecasting in C is a way to convert a variable from one data type to another. This is necessary when performing operations that require variables to be of the same type, or when a function expects a parameter of a certain type. Typecasting can be **implicit** or **explicit**.

CHP#15: OPERATORS & EXPRESSIONS

- **Type Casting:**

Typecasting in C is a way to convert a variable from one data type to another. This is necessary when performing operations that require variables to be of the same type, or when a function expects a parameter of a certain type. Typecasting can be **implicit** or **explicit**.

- **Implicit Typecasting (Automatic Type Conversion):**

Implicit typecasting, also known as automatic type conversion, is performed by the compiler without any explicit request from the programmer. This usually happens when you perform operations between variables of different types, and the compiler automatically converts one operand to the type of the other.

CHP#15: OPERATORS & EXPRESSIONS

- **Type Casting:**

Typecasting in C is a way to convert a variable from one data type to another. This is necessary when performing operations that require variables to be of the same type, or when a function expects a parameter of a certain type. Typecasting can be **implicit** or **explicit**.

- **Implicit Typecasting (Automatic Type Conversion):**

Implicit typecasting, also known as automatic type conversion, is performed by the compiler without any explicit request from the programmer. This usually happens when you perform operations between variables of different types, and the compiler automatically converts one operand to the type of the other.

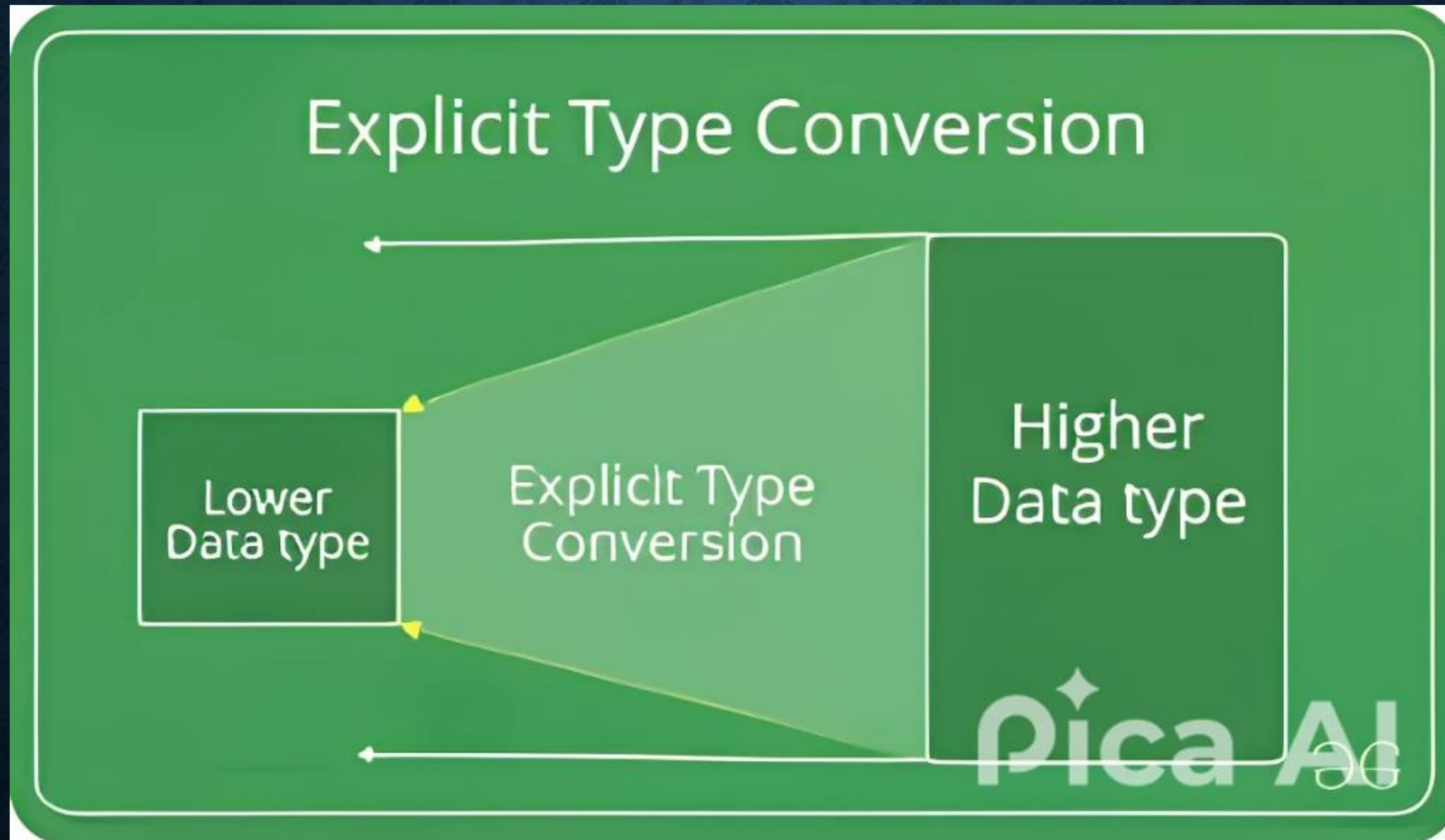
- **Explicit Typecasting (Manual Type Conversion):**

Explicit typecasting, or manual type conversion, is performed by the programmer using the typecast operator. This is necessary when you need to force a conversion from one type to another.

- **Syntax:**

- (type) expression

CHP#15: OPERATORS & EXPRESSIONS



CHP#15: OPERATORS & EXPRESSIONS

- **Examples of Explicit Data Type Casting:**

=> `int a = 5, b = 2;`

=> `float result = a / b; // Converts a to float`

Without casting:

`5 / 2 = 2`

CHP#15: OPERATORS & EXPRESSIONS

- **Examples of Explicit Data Type Casting:**

=> `int a = 5, b = 2;`

=> `float result = a / b; // Converts a to float`

Without casting:
 $5 / 2 = 2$

=> `int a = 5, b = 2;`

=> `float result = (float)a / b; // Converts a to float`

With casting:
 $(\text{float})5 / 2 = 2.5$

CHP#13: ALGORITHM & FLOWCHARTS

Any Doubt?

Topic#03:

Some common Library Functions

CHP#15: OPERATORS & EXPRESSIONS

- Some Commonly used Library Functions:

Header File	Function	Description	Example
stdio.h	printf()	Output text to screen	printf("Hello");
stdio.h	scanf()	Input from user	scanf("%d", &a);

CHP#15: OPERATORS & EXPRESSIONS

- Some Commonly used Library Functions:

Header File	Function	Description	Example
math.h	sqrt()	Square root	sqrt(16); → 4
math.h	pow()	Power function	pow(2,3); → 8
math.h	abs()	Absolute value	abs(-5); → 5

CHP#15: OPERATORS & EXPRESSIONS

- Some Commonly used Library Functions:

Header File	Function	Description	Example
string.h	strlen()	Length of string	strlen("C"); → 1
string.h	strcpy()	Copy string	strcpy(a,b);
string.h	strcmp()	Compare strings	strcmp(a,b);

CHP#15: OPERATORS & EXPRESSIONS

- Some Commonly used Library Functions:

Header File	Function	Description	Example
ctype.h	toupper()	Convert char to uppercase	toupper('a');
ctype.h	tolower()	Convert char to lowercase	tolower('A');

CHP#15: OPERATORS & EXPRESSIONS

- Some Commonly used Library Functions:

Header File	Function	Description	Example
stdlib.h	malloc()	Allocate memory	malloc(10);
stdlib.h	free()	Deallocate memory	free(ptr);
stdlib.h	exit()	Terminate program	exit(0);

CHP#15: OPERATORS & EXPRESSIONS

- Some Commonly used Library Functions:

Header File	Function	Description	Example
stdio.h	printf()	Output text to screen	printf("Hello");
stdio.h	scanf()	Input from user	scanf("%d", &a);
math.h	sqrt()	Square root	sqrt(16); → 4
math.h	pow()	Power function	pow(2,3); → 8
math.h	abs()	Absolute value	abs(-5); → 5
string.h	strlen()	Length of string	strlen("C"); → 1
string.h	strcpy()	Copy string	strcpy(a,b);
string.h	strcmp()	Compare strings	strcmp(a,b);
ctype.h	toupper()	Convert char to uppercase	toupper('a');
ctype.h	tolower()	Convert char to lowercase	tolower('A');
stdlib.h	malloc()	Allocate memory	malloc(10);
stdlib.h	free()	Deallocate memory	free(ptr);
stdlib.h	exit()	Terminate program	exit(0);

CHP#13: ALGORITHM & FLOWCHARTS

Any Doubt?

CHP#15: OPERATORS & EXPRESSIONS

- **HomeWork Task:**
- Search more common library functions using textbook/Internet.
- Explore Ternary Operator (like how it works, how you can use, is this any other implementation of ternary operator or not.)